Análise técnica de backend · 30 de outubro de 2026

Relatório de análise de backend

Controle de Gastos / Verbas

Repositório local: `controle_gastos`

Branch: **main** @ `0629907` (local atrás de `origin/main` em 7 commits)

Stack observada: Java 25 / Spring Boot 4.1 / Spring Modulith 2.1 / PostgreSQL / Flyway / JWT HS256

Objetivo: auditar arquitetura, APIs, regras de negócio, segurança, dados, testes e operação do backend — somente leitura.

Escopo desta entrega: análise + PDF. Issues no GitHub somente após aprovação explícita por ID (`BE-XXX`). Nenhum código, branch, commit ou PR foi alterado.

1. Resumo executivo

O backend já entrega o núcleo financeiro do produto pessoal: identidade (cadastro/login, refresh rotativo, MFA TOTP, OAuth Google/GitHub), renda mensal, verbas com cinco naturezas, ledger com saldo negativo permitido, histórico, relatórios CSV/XLSX e limites Modulith verificáveis. Money BRL com `BigDecimal`/escala 2 está bem centralizado.

Principais riscos / lacunas:

1. **P1** A regra confirmada “soma das verbas-base ≤ renda” não é aplicada na criação/edição de verba — só bloqueia redução de renda.
2. **P1** Hash de senha usa BCrypt, em desacordo com ADR-005 / requisitos (Argon2id).
3. **P1** Ausência de fail-fast quando `AUTH_JWT_SECRET` (e chave TOTP) estão vazios: gera segredo efêmero em memória.
4. **P1** Módulos `privacy`, `sharing`, `audit` e `notifications` são stubs — bloqueiam abertura pública e a feature de compartilhamento.
5. **P2** Três testes de integração de saldo/meta falharam nesta execução por datas fixas no passado vs. `Clock.systemUTC()` atual (out/2026).

Não há P0 de exploração ativa comprovada nesta auditoria estática. O sistema está maduro para uso privado controlado, mas não para cadastro público sem fechar LGPD, Argon2 e a invariante renda/verbas.

2. Escopo e metodologia

Inspecionado

- `docs/ideas/fundacao-tecnica.md`, ADRs, `docs/privacy/`, `docs/api/openapi.yaml`, README
- Módulos Modulith: identity, income, envelopes, ledger, reporting (+ stubs)
- SecurityFilterChain, sessão JWT, rate limit HMAC, OAuth clients, Money, Flyway V1–V12
- Controllers REST e serviços de autorização por recurso

Comandos executados

- `git status --short --branch`
- Gradle 9.5 (cache local): `gradle -p backend test --no-daemon`
- Resultado: **161 testes, 3 falhas** (GoalProgress ×2, SavingsTarget ×1). Docker/Testcontainers disponível nesta execução.

Limitações

- Auditoria somente leitura; sem exploração ofensiva, load test ou revisão de produção.
- `.env` e segredos reais não foram lidos.

- `gh auth` com token inválido — issues não podem ser criadas até reautenticação + aprovação por ID.
- Branch local atrás de origin em 7 commits; achados referem-se ao HEAD local.
- reportlab não instalado; PDF gerado via HTML + Chrome headless.

3. Visão geral da arquitetura

Monólito modular Spring Modulith, processo único, PostgreSQL + Flyway, sem filas/cache (alinhado à fundação). Dependências entre módulos respeitam boundaries (`ModularityTest`). Fluxos críticos:

1. **Identidade:** register/login → JWT access + refresh opaco em cookie; MFA e OAuth opcionais; SessionValidationFilter consulta sessão ativa.
2. **Renda:** mudança no mês corrente (America/Sao_Paulo); histórico paginado; constraint via `IncomeChangeConstraint`.
3. **Verbas:** CRUD/arquivo; autorização owner vs. participante na consulta; participantes persistidos em tabela, sem API de convite.
4. **Ledger:** despesas/aportes, saldo acumulável, histórico, edição/exclusão soft-delete, summary e relatórios.

> **Pontos fortes:** Money imutável com `RoundingMode.UNNECESSARY`; open-in-view=false; actuator limitado a health/info; cookies refresh bem documentados; testes de integração amplos em identity/income/reporting.

4. Matriz priorizada de achados

ID	Título	Pri	Sev	Tipo	Esforço	Confiança
BE-001	Criação/edição de verba não valida soma-base ≤ renda	P1	Alta	Evidência estática	M	Alta
BE-002	Hash de senha BCrypt em vez de Argon2id (ADR-005)	P1	Alta	Evidência estática	M	Alta
BE-003	Segredos JWT/TOTP vazios geram chave efêmera (sem fail-fast)	P1	Alta	Risco plausível	S	Alta
BE-004	Stubs privacy/sharing/audit/notifications vs. requisitos	P1	Alta	Evidência estática	XL	Alta
BE-005	Testes de saldo/meta acoplados ao calendário real	P2	Média	Teste falho reproduzido	S	Alta
BE-006	Listagem de lançamentos por verba não exclui soft-delete	P2	Média	Evidência estática	XS	Alta
BE-007	Edição/exclusão de gasto autoriza por owner da verba, não author	P2	Média	Risco / inconsistência produto	S	Média
BE-008	Provisão mensal de gasto anual fora do mínimo de renda	P2	Média	Evidência estática	M	Alta
BE-009	RestClient OAuth sem timeout explícito	P2	Média	Risco plausível	S	Média
BE-010	GET /envelopes expõe available = baseAmount (saldo real só no summary)	P3	Baixa	Contrato / DX	S	Alta
BE-011	usagePct do summary usa double derivado de Money	P3	Baixa	Evidência estática	XS	Alta
BE-012	Optimistic lock (@Version) exposto sem If-Match no PATCH	P3	Baixa	Hipótese	S	Média

5. Detalhamento dos achados

BE-001 — Criação/edição de verba não valida soma-base ≤ renda

P1 Alta M · Categoria: regras de negócio · Confiança: alta

Localização: `EnvelopeService.create/update`; domínio `MonthlyAllocationPlan` / `AllocationExceedsIncomeException` (handler existe, caminho de serviço não usa); `EnvelopeIncomeConstraint` só alimenta redução de renda.

Problema: Comentário em create afirma delegação à constraint de renda, mas a constraint só impede baixar a renda abaixo da soma já alocada. É possível criar/aumentar verbas além da renda configurada.

Impacto: Quebra invariante central da fundação técnica; unallocated e alertas ficam inconsistentes com a intenção do produto.

Recomendação: Antes de salvar create/update, carregar renda efetiva do mês (via porta já usada pelo ledger) e aplicar `MonthlyAllocationPlan.create` (ou equivalente). Manter a constraint na redução de renda.

Aceite: Tentativa de criar verba que exceda a renda retorna 4xx com problema de alocação; testes de integração cobrindo create e update; redução de renda continua bloqueada.

Decisão sugerida: corrigir.

BE-002 — Hash de senha BCrypt em vez de Argon2id

P1 Alta M · Categoria: segurança · Confiança: alta

Localização: `AuthConfiguration.passwordEncoder()` → `BCryptPasswordEncoder(12)`; ADR-005 e `docs/privacy/requisitos-de-seguranca.md` exigem Argon2id com parâmetros explícitos.

Impacto: Divergência documentada vs. implementação; custo de cracking diferente do perfil aceito; migração futura exige rehash no login.

Recomendação: Trocar para Argon2id (parâmetros do ADR), com migração transparente no login (detectar prefixo BCrypt e regravar). Benchmark na VPS conforme operação de autenticação.

Aceite: Novos hashes Argon2id; login de contas BCrypt existentes funciona e atualiza hash; testes unitários/integração.

Decisão sugerida: corrigir antes de abertura pública.

BE-003 — Segredos JWT/TOTP vazios geram chave efêmera

P1 Alta S · Categoria: segurança / deploy · Confiança: alta

Localização: `AuthConfiguration.jwtSecretBytes / totpEncryptionKeyBytes` — se blank, usa `SecureRandom` em memória.

Problema: Em ambiente “quase produção” sem variável, a API sobe; tokens e TOTP ficam inválidos após restart ou entre instâncias.

Recomendação: Fail-fast em perfis não locais quando secrets ausentes; manter geração efêmera só em profile de teste/local documentado.

Aceite: Boot com profile prod sem secret falha; teste de configuração; local/test continua funcional.

Decisão sugerida: corrigir.

BE-004 — Módulos privacy / sharing / audit / notifications vazios

P1 Alta XL · Categoria: arquitetura / produto / LGPD · Confiança: alta

Localização: apenas `package-info.java` em cada módulo; README confirma ausência de compartilhamento e push.

Impacto: Cadastro público bloqueado pelos requisitos de privacidade; proposta de verbas compartilhadas sem superfície de API; auditoria detalhada adiada (ADR-009) permanece gap consciente.

Recomendação: Tratar como épicos separados com ADR quando iniciar. Priorizar privacy (export/delete/canal) antes de cadastro aberto; sharing com autorização owner-only para convites.

Aceite: Endpoints e testes por épico; inventário de dados atualizado.

Decisão sugerida: planejar / adiar conforme marco (privado vs. público).

BE-005 — Testes de saldo/meta acoplados ao calendário

P2 Média S · Categoria: testes · Confiança: alta · **Defeito reproduzido**

Evidência: Em 2026-10-30, `SavingsTargetApiIntegrationTest` esperava available `1050.00` em `month=2026-09` e obteve `0.00`. GoalProgress: planned/percent zerados no mês fixo passado. Causa: envelope criado com `clock.instant()` (outubro); `availableAt`/`goalProgressFor` retornam zero se o mês consultado é anterior à criação.

Recomendação: Congelar Clock nos testes de integração (bean de teste) ou usar mês derivado de `YearMonth.now(BUSINESS_ZONE)`.

Aceite: Suite verde independentemente da data de execução.

Decisão sugerida: corrigir.

BE-006 — Listagem de entries não filtra soft-delete

P2 Média XS · Categoria: API / dados · Confiança: alta

Localização: `LedgerEntryRepository.findFiltered` sem `deletedAt IS NULL`; somatórios e history já excluem excluídos.

Impacto: GET `/envelopes/{id}/entries` pode exibir gastos logicamente excluídos, divergindo do saldo.

Recomendação: Filtrar deleted por padrão; opcional includeDeleted alinhado ao history.

Aceite: Teste de integração: após DELETE, lista da verba não retorna o item; saldo inalterado já coberto.

Decisão sugerida: corrigir.

BE-007 — Autorização de edição/exclusão por owner da verba

P2 Média S · Categoria: autorização · Confiança: média

Localização: `LedgerService.editExpense/deleteExpense` comparam `entry.ownerId()` (dono da verba) ao usuário atual. README menciona edição/exclusão de “lançamentos próprios”.

Impacto: Quando sharing existir, participante autor não edita/exclui o próprio gasto; só o dono. Pode ser intencional (controle do criador) — precisa decisão explícita.

Recomendação: Documentar regra no ADR/OpenAPI; se “próprios” = authorId, ajustar checagem e testes.

Decisão sugerida: investigar / decidir regra antes de sharing.

BE-008 — Gasto anual MONTHLY fora do mínimo de renda

P2 Média M · Categoria: regras de negócio · Confiança: alta

Localização: `EnvelopeIncomeConstraint` usa `sumBaseAmounts`; ANNUAL_EXPENSE força `base_amount = 0`.

Impacto: Parcelas mensais de provisão anual não entram no piso da renda; usuário pode reduzir renda abaixo do comprometido.

Recomendação: Incluir provisão do mês corrente das annual MONTHLY no cálculo de mínimo (e na validação de create, com BE-001).

Decisão sugerida: corrigir junto com BE-001.

BE-009 — RestClient OAuth sem timeout

P2 Média S · Categoria: confiabilidade · Confiança: média

Localização: `GoogleOAuthClient` / `GitHubOAuthClient` com `RestClient.create()` padrão.

Recomendação: Timeouts de connect/read explícitos; manter sem retry no exchange de code (já documentado).

Decisão sugerida: corrigir.

BE-010 — available enganoso em GET /envelopes

P3 Baixa · Comentário no controller admite available = baseAmount para evitar ciclo Modulith; saldo real em `/ledger/summary`.

Recomendação: Remover campo available da resposta de envelopes ou documentar OpenAPI como “base do mês, não saldo”; clientes devem usar summary.

Decisão sugerida: mitigar (contrato).

BE-011 — usagePct com double

P3 Baixa · `LedgerController` calcula percentual com `doubleValue()`. Não é saldo persistido, mas foge da disciplina Money.

Recomendação: Percentual com BigDecimal (escala definida) ou string decimal; alinhar OpenAPI.

Decisão sugerida: corrigir quando tocar no summary.

BE-012 — @Version sem controle de concorrência na API

P3 · Version é serializado, mas PATCH não exige versão do cliente. Lost updates possíveis em edição concorrente de verba.

Decisão sugerida: medir/adiar até multi-dispositivo intensivo; ou aceitar If-Match.

6. Segurança e integridade dos dados

- **Autenticação:** JWT curto + validação de sessão; MFA com desafio opaco; recovery codes hasheados; TOTP cifrado AES-GCM — maduro para uso privado.
- **Autorização por recurso:** envelopes usam owner/participante; ledger register restringe CONTRIBUTION ao owner; IDOR básico mitigado por not-found/forbidden.
- **Rate limiting:** AuthAttemptService com HMAC de e-mail/IP — alinhado à privacidade.
- **CSRF desabilitado:** esperado para API Bearer + cookie SameSite Strict no path de auth; manter origins explícitos.
- **Gaps:** BE-002, BE-003, privacy stub, cadastro sem verificação imediata de e-mail (já documentado).
- **Integridade Money:** domínio sólido; exceção do usagePct (BE-011).
- **Soft-delete:** inconsistência na listagem (BE-006).

7. APIs, testes, confiabilidade e operação

APIs

- Prefixo `/api/v1`; OpenAPI presente; validação manual via JsonNode em vários controllers (consistente, mas verbosa).
- Paginação com limites (size ≤ 100) em history/income/entries.
- Problemas HTTP via ProblemDetail em handlers de módulo.

Testes

- 161 testes; 3 falhas reproduzidas (BE-005). Identity, income, reporting, modularity e grande parte do ledger passaram.
- Boa cobertura de MFA/OAuth/password-reset; falta cobertura da invariante renda×verbas no create (BE-001).

Confiabilidade / operação

- Health probes habilitados; mail com timeouts SMTP; IdentityRetentionService agendado.
- OAuth sem timeout HTTP (BE-009).
- Compose/Nginx/ADR-012; CI valida presença de secrets no workflow (não fail-fast da app).

8. O que não recomendo fazer agora

- Microserviços, Kafka, Redis/cache, CQRS ou observabilidade pesada — fora da fundação e sem evidência de necessidade.
- Reintroduzir Keycloak sem novo ADR.
- Refatoração ampla “Clean Architecture” — o Modulith atual é coerente.

- Otimizações prematuras de N+1 no summary sem medição (hipótese apenas).
- Abrir cadastro público antes de privacy + Argon2 + BE-001.

9. Plano de execução sugerido

1. **Curto prazo:** BE-005 (suite verde), BE-006 (soft-delete na lista), BE-003 (fail-fast de secrets).
2. **Núcleo financeiro:** BE-001 + BE-008 (invariante renda/alocação completa).
3. **Antes de terceiros:** BE-002 (Argon2), épico privacy (parte de BE-004).
4. **Produto compartilhado:** decidir BE-007, depois sharing (BE-004).
5. **Higiene:** BE-009, BE-010, BE-011 conforme oportunidade.

10. Decisões solicitadas

Quais achados você aprova para criação de issues? Responda com IDs explícitos (ex.: `BE-001`, `BE-005`). Nenhum outro será criado sem autorização.

> **Bloqueio atual:** `gh auth status` reporta token inválido para a conta GitHub. Após aprovar IDs, será necessário `gh auth login` (ou renovar token) antes de criar issues.

Sugestão inicial de lote de alto valor / esforço controlado: **BE-001, BE-003, BE-005, BE-006**. BE-002 e privacy (BE-004) se o marco for abertura a terceiros.

11. Apêndice — referências

- `backend/src/main/java/br/com/controlegastos/envelopes/application/EnvelopeService.java`
- `backend/src/main/java/br/com/controlegastos/envelopes/application/EnvelopeIncomeConstraint.java`
- `backend/src/main/java/br/com/controlegastos/identity/config/AuthConfiguration.java`
- `backend/src/main/java/br/com/controlegastos/identity/config/SecurityConfiguration.java`
- `backend/src/main/java/br/com/controlegastos/ledger/application/LedgerService.java`
- `backend/src/main/java/br/com/controlegastos/ledger/infrastructure/LedgerEntryRepository.java`
- `backend/src/main/java/br/com/controlegastos/ledger/web/LedgerController.java`
- `docs/decisions/0005-autenticacao-propria-por-email-e-senha.md`
- `docs/privacy/requisitos-de-seguranca.md`
- JUnit XML: falhas em SavingsTarget / GoalProgress sob `backend/build/test-results/test/`

Relatório gerado em modo somente leitura. Fatos, riscos e hipóteses foram separados nos campos “Tipo” de cada achado. Sem segredos, tokens ou dados pessoais.